

Pioneer Delivery Company's Algorithm Portfolio

Part I: Greedy Algorithms

Project Overview

Pioneer Delivery is a growing local delivery company that receives many delivery requests each day. Each delivery occupies a driver for a specified time interval.

For example, a request may require a driver from 9:10 AM until 9:45 AM. A driver cannot handle two overlapping deliveries.

Pioneer Delivery would like software that can answer two basic operational questions:

1. If only one driver is available, what is the largest number of delivery requests that driver can complete?
2. If Pioneer wants to accept every delivery request, what is the minimum number of drivers needed?

You have been hired to implement the first version of the company's scheduling system. This project is Part I of the semester-long **Pioneer Delivery Algorithm Portfolio**. In later parts of the project, Pioneer will face problems involving divide-and-conquer, dynamic programming, and graph algorithms.

Delivery Data

Each delivery request contains at least:

```
id
start_time
finish_time
```

For example:

```
D01  10  13
D02   2   5
D03   4   7
D04   1   8
D05   8  11
D06  11  14
D07  13  16
```

You may assume that:

- `start_time < finish_time`;
- times are represented by integers;

- if one delivery finishes at time t and another starts at time t , the same driver may perform both deliveries.

Your program should read a collection of delivery requests from a file.

You may use a programming language of your choice.

Task A: Maximum Deliveries for One Driver

[20 points]

Suppose Pioneer Delivery has only one driver available.

Given n delivery requests, design and implement a greedy algorithm that selects the **maximum possible number of mutually compatible deliveries**.

Your program should output:

- the selected delivery IDs;
- their start and finish times;
- the total number of deliveries completed.

Your algorithm should run in

$$O(n \log n)$$

time, including sorting.

In your README, briefly explain:

1. What is the greedy choice made by your algorithm?
2. Why does this greedy strategy produce an optimal solution?
3. What is the running time of your algorithm?

Required Test Cases

At minimum, test your implementation on the following cases.

Test	Delivery Intervals	Maximum Selected
1	No deliveries	0
2	(1, 3)	1
3	(1, 3), (3, 5), (5, 7)	3
4	(1, 5), (2, 6), (3, 7)	1
5	(5, 7), (1, 3), (3, 5), (2, 4)	3
6	(1, 4), (2, 4), (4, 6)	2

For cases in which more than one optimal schedule exists, your program may output any optimal schedule.

Task B: Minimum Number of Drivers

[25 points]

Pioneer Delivery now wants to accept **all** delivery requests.

Design and implement a greedy algorithm that determines the **minimum number of drivers required** so that every delivery can be completed.

Your program should also assign each delivery to a driver and output the resulting schedules.

For example:

Driver 1: D1 D4 D7

Driver 2: D2 D5

Driver 3: D3 D6

Minimum drivers required: 3

Your implementation should use a **priority queue (min-heap)** and should run in

$$O(n \log n).$$

In your README, briefly explain:

1. What information is stored in the priority queue?
2. How is a driver selected for each new delivery?
3. Why does this algorithm use the minimum possible number of drivers?
4. What is the running time?

Required Test Cases

Test	Delivery Intervals	Minimum Drivers
1	No deliveries	0
2	(1, 3)	1
3	(1, 3), (3, 5), (5, 7)	1
4	(1, 5), (2, 6), (3, 7)	3
5	(1, 4), (2, 5), (4, 7), (5, 8)	2
6	(5, 8), (1, 4), (2, 5), (4, 6), (6, 9)	2

Your exact assignment of deliveries to drivers may differ from another correct solution, but the number of drivers must be minimum and no driver may be assigned overlapping deliveries.

Task C: Interview-Style Variant

[20 points]

Pioneer learns that some delivery requests may need to be rejected because only one driver is available.

Given n delivery intervals, determine the **minimum number of deliveries that must be rejected** so that all remaining deliveries are mutually compatible.

Design and implement an algorithm that runs in

$$O(n \log n).$$

In your README, briefly explain:

1. How is this problem related to Task A?
2. If the maximum number of compatible deliveries is k , how many deliveries must be rejected?
3. What is the running time of your algorithm?

Interview connection. This problem is equivalent to the main idea in LeetCode 435, *Non-overlapping Intervals*. You are encouraged to try the corresponding LeetCode problem after completing your own implementation.

Required Test Cases

Test	Delivery Intervals	Minimum Rejected
1	No deliveries	0
2	(1, 3)	0
3	(1, 3), (3, 5), (5, 7)	0
4	(1, 5), (2, 6), (3, 7)	2
5	(5, 7), (1, 3), (3, 5), (2, 4)	1
6	(1, 2), (2, 3), (3, 4), (1, 3)	1

Task D: When a Greedy Choice Fails

[15 points]

Pioneer Delivery also handles flexible same-day delivery requests.

Each delivery j :

- takes exactly one unit of driver time;
- is ready starting at time R_j , where R_j is a nonnegative integer;
- has a profit $P_j > 0$, where $P_i \neq P_j$ for $i \neq j$.

A schedule S assigns each delivery j to a time slot $T_j(S)$ such that

$$R_j \leq T_j(S)$$

and

$$T_j(S) \neq T_i(S) \quad \text{for } i \neq j.$$

For each delivery j , define its delay as

$$D_j(S) = T_j(S) - R_j.$$

The value Pioneer receives from delivery j is

$$V_j(S) = \frac{P_j}{1 + D_j(S)},$$

and the goal is to find a schedule maximizing

$$V(S) = \sum_j V_j(S).$$

A manager proposes the following greedy strategy G :

Consider the deliveries in order of decreasing profit. For each delivery, schedule it in the earliest available time slot at or after its ready time.

Show, by means of a **counterexample**, that this greedy strategy does **not** always produce an optimal schedule.

Your answer should:

1. give the ready time R_j and profit P_j for each delivery;
2. show the schedule produced by greedy strategy G and compute $V(G)$;
3. give a better schedule S and compute $V(S)$;
4. explain why this proves that G is not always optimal.

Hint: A counterexample with only a few deliveries is sufficient.

Submission

[5 points]

Submit all source code, a data file, and a `README.md`.

Your `README` should include:

- the programming language you used;
- instructions for compiling and running your program;
- the input-file format as well as a test data file;
- a brief description of the organization of your program;
- the algorithm explanations requested in Tasks A–C;
- the results of the required test cases.

Organize your code into meaningful functions, classes, or modules rather than placing the entire implementation in one main function.

GitHub Portfolio Recommendation

You are encouraged to maintain this project in a GitHub repository and continue using the same repository for later parts of the Pioneer Delivery Algorithm Portfolio.

A well-organized repository with a clear README can serve as a project that you can discuss in technical interviews or include in your portfolio.

Using GitHub is recommended but is **not required** for grading.